

Introduction to R

Open slides

Get R

From CRAN

Rstudio

Ubuntu: `sudo apt install r-base`

Some Introduction Material

From the developers: Introduction to R

Rstudio: Documentation

R for Data Science

Background

- R is a general programming language with a specialization for statistics
- R is generally being used through a REPL and interpreted live
- Functional Programming paradigm: All about `objects` and `functions`
- Major Appeals:
 - Extremely large ecosystem of packages, contributors, support system and companies
 - Many general and special solutions already available in R
 - Easy to learn and get going
 - Amongst the best graphics/plotting ecosystem for scientific output
- Major Drawbacks:
 - Functional Programming lends itself to “scripting” without structure
 - Barrier for introducing structure through packages high
 - R is an interpreted and not compiled language: no optimization
 - “Always copy paradigm” makes R slow in many settings
 - R is old and its successors already exist

Basics

Assigning and manipulating variables

Note: R has 2 assignment operator: `<-` and `=`. For all relevant purposes of the tutorial, we can safely use either of those operators interchangeably.

```
#####  
### Assigning and manipulating variables ###
```

```
#####
```

```
x = 5  
print(x)
```

```
[1] 5
```

```
# just giving the variable name, will print it to the REPL by default  
x
```

```
[1] 5
```

```
# assign value from one variable to the next  
y = x  
  
# object did not trigger a copy of "x", rather y "points" to x  
print(y)
```

```
[1] 5
```

```
# change the value of x  
x = 3  
  
print(x)
```

```
[1] 3
```

```
# at the moment we changed the value of "x", a copy of the old "x" was triggered  
and assigned to "y"  
print(y)
```

```
[1] 5
```

Working with data frames

One of the most important object types in R are `data.frames`. They are rectangular tables with rows and columns, whereas the column can have different `types`, which is the main differentiator to a `matrix`, where the column types are all identical.

```
#####
### Assigning and manipulating variables ###
#####

dat = data.frame(
  name = c("Adam", "Paul", "Elly"),
  age = c(22, 43, 54)
)

# look at general structure
str(x)
```

```
num 3
```

```
# extracting single column and storing in a vector object
names = dat$name

# can also be accessed with the bracket operator
ages = dat[, "age"]

str(names)
```

```
chr [1:3] "Adam" "Paul" "Elly"
```

```
str(ages)
```

```
num [1:3] 22 43 54
```

```
# subsetting
dat[dat$name == "Paul",]
```

```
  name age
2 Paul  43
```

```
dat[dat$age %in% c(22, 54),]
```

```

  name age
1 Adam  22
3 Elly  54

```

```

# adding a column to a data frame
dat$id = 1:nrow(dat)
dat[, "id_2"] = dat$id + 2

print(dat)

```

```

  name age id id_2
1 Adam  22  1    3
2 Paul  43  2    4
3 Elly  54  3    5

```

```

# conditional assignment
dat$id[dat$name == "Adam"] <- 12

print(dat)

```

```

  name age id id_2
1 Adam  22 12    3
2 Paul  43  2    4
3 Elly  54  3    5

```

Reading and Writing Data from/to files

Most of the time, data is being input to R from files. However, R has interfaces to most data storage systems out there, including all databases and cloud storages.

```

#####
### Read and Write data ###
#####

# create data frame
dat = data.frame(
  name = c("Adam", "Paul", "Elly"),
  age = c(22, 43, 54)
)
str(dat)

```

```
'data.frame':  3 obs. of  2 variables:
 $ name: chr  "Adam" "Paul" "Elly"
 $ age : num  22 43 54
```

```
# write data to disk as csv
write.table(dat, file = "my_data.csv", row.names = FALSE, sep = ",")

# read file back in
dat_read = read.table(file = "my_data.csv", sep = ",", header = TRUE)
str(dat_read)
```

```
'data.frame':  3 obs. of  2 variables:
 $ name: chr  "Adam" "Paul" "Elly"
 $ age : int  22 43 54
```

Vectors and Matrices

In R, everything is a **vector** or a **list** or an **array** of vectors. A **data.frame** is a list of vectors of different types, a **matrix** is a list of vectors of a single type. A **scalar** is a vector of length 1.

```
#####
### Vectos and Matrices ###
#####

# integer vector of length one
x = as.integer(1)
str(x)
```

```
int 1
```

```
is.vector(x)
```

```
[1] TRUE
```

```
# numeric vector
y = rnorm(10)
str(x)
```

```
int 1
```

```
x
```

```
[1] 1
```

```
# matrix
D = matrix(data = rnorm(5*5), nrow = 5, ncol = 5)
D
```

```
      [,1]      [,2]      [,3]      [,4]      [,5]
[1,]  0.2196859 -1.895069977  2.5102935  0.5114221 -0.7414099
[2,]  1.6448779  1.196900565 -1.5475604 -0.0776841  0.5295905
[3,] -0.5247454  0.004227332  0.1039766 -0.1071041 -2.6396135
[4,] -0.6808976 -0.797034801  1.2354847  1.4583665 -0.1033763
[5,]  0.7009601 -0.043234149  0.5780782  0.8039246  0.1107747
```

```
class(D)
```

```
[1] "matrix" "array"
```

```
# specialized functions can be applied to matrices
means = colMeans(D)
means
```

```
[1]  0.2719762 -0.3068422  0.5760545  0.5177850 -0.5688069
```

```
# extract diagonal
d = diag(D)
d
```

```
[1] 0.2196859 1.1969006 0.1039766 1.4583665 0.1107747
```

Data Exploration and Visualization

Getting a good overview and understanding of the data sets we work with is very important. Summary statistics and visualizations are important tools here. We will be using `dlookr` and `ggplot2`.

```
#####
### Data Exploration and Visualization ###
#####

if(!require(dlookr)) install.packages('dlookr')
if(!require(ggplot2)) install.packages('ggplot2')
library(dlookr, ggplot2)

# load a public data set
data(iris)
str(iris)
```

```
'data.frame':  150 obs. of  5 variables:
 $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
 $ Species      : Factor w/ 3 levels "setosa","versicolor",...: 1 1 1 1 1 1 1 1 1 1 ...
1 1 ...
```

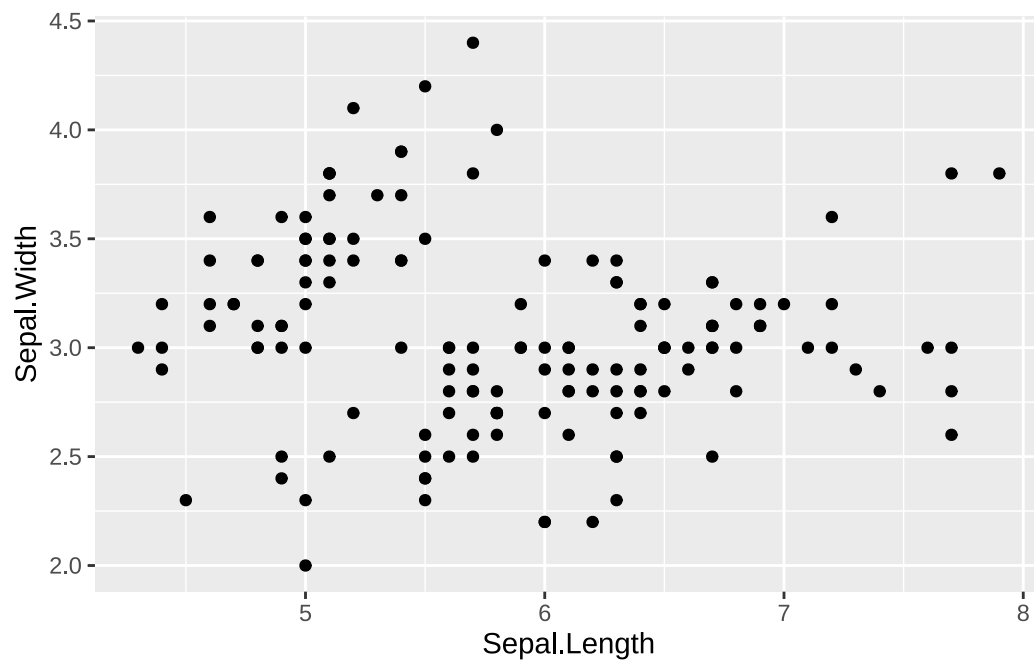
```
# broad overview of data set
dlookr::diagnose(iris)
```

```
# A tibble: 5 × 6
```

	variables	types	missing_count	missing_percent	unique_count	unique_rate
	<chr>	<chr>	<int>	<dbl>	<int>	<dbl>
1	Sepal.Length	numeric	0	0	35	0.233
2	Sepal.Width	numeric	0	0	23	0.153
3	Petal.Length	numeric	0	0	43	0.287
4	Petal.Width	numeric	0	0	22	0.147
5	Species	factor	0	0	3	0.02

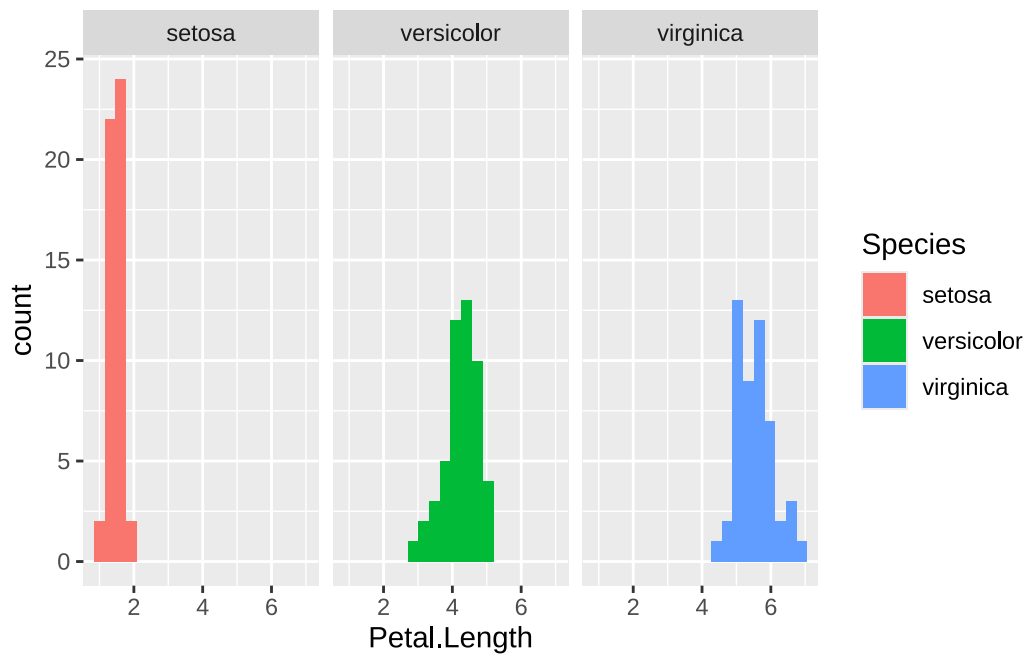
```
# plot our data
# scatterplot
p = ggplot(iris, aes(x = Sepal.Length, y = Sepal.Width)) +
  geom_point()

p
```



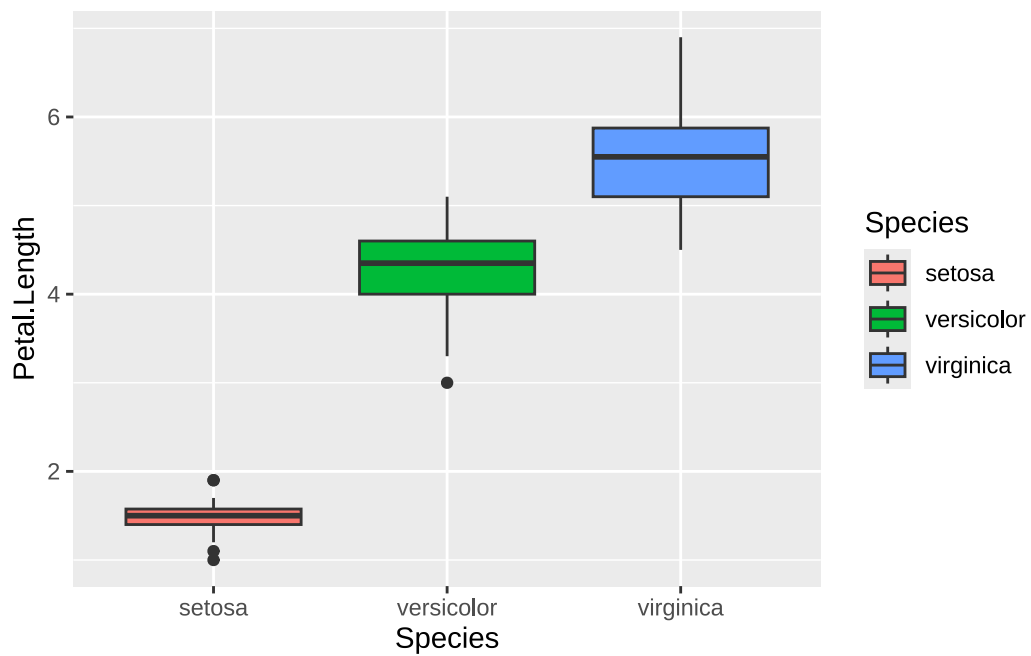
```
# histogram for temp by month
p2 = ggplot(iris, aes(x = Petal.Length)) +
  geom_histogram(aes(fill = Species), bins = 20) +
  facet_wrap(~ Species)
```

p2



```
# boxplot
p3 = ggplot(iris, aes(x = Species, y = Petal.Length, fill = Species)) +
  geom_boxplot()

p3
```



Writing Functions

All operators in R are functions, for instance `+` is just a special way of calling a function:

```
# get help for a function
?sum

# get help for the function '+'
?`+`

# check the function definition for `+`
fix(`+`)
```

A function is an operation on input variables ($f(x)$), the function can be anything interpretable by R. For example, this is how we can define a function that computes and returns the empirical mean (average) of a numeric vector:

```
my_mean_function = function(y) {

  n = length(y)
  sum = 0
  for(i in 1:n) sum = sum + y[i]

  mean = sum / n

  return(mean)

}
```

We can now use that function inside R for estimating means on any input vector `y` for which the operators `+` and `/` are defined.

```
# generate random vector
x = rnorm(1000, mean = 23.42, sd = 4)

# calculate mean with our function
our_mean = my_mean_function(x)
our_mean
```

```
[1] 23.42036
```

```
# compare with R's built in function
mean(x)
```

[1] 23.42036